

CODE QUALITY

Introduction to Code Quality

Code quality refers to how well-written, maintainable, and reliable a piece of code is. High-quality code meets specific requirements while being easy to understand, extend, test, and maintain over time. The importance of code quality becomes more significant as software projects grow in complexity, with more developers contributing to the same codebase. Good code quality can reduce the number of bugs, simplify updates, and ultimately result in better-performing software.

Key Aspects of Code Quality:

1. **Readability:** Code should be easy to read and understand. Clear naming conventions, proper indentation, and appropriate comments make code easier to follow.
2. **Maintainability:** Code should be simple to update or fix. This means structuring the code logically and minimizing dependencies between components.
3. **Efficiency:** High-quality code is optimized for performance. It minimizes resource usage and is designed to be scalable.
4. **Testability:** Code should be written in such a way that it can be easily tested. This involves modular design, where components can be isolated and tested individually.
5. **Reusability:** Quality code avoids redundancy by reusing components whenever possible. This reduces development time and minimizes potential errors.
6. **Robustness:** Code should be resistant to errors or failures. This involves proper error handling, input validation, and exception management.
7. **Compliance with Standards:** High-quality code follows coding standards and best practices, ensuring consistency across a project and adherence to industry standards.

Benefits of Code Quality:

- **Fewer Bugs:** Cleaner, more organized code is less prone to errors.
- **Easier Collaboration:** Consistent, readable code allows developers to work together efficiently.
- **Reduced Technical Debt:** High-quality code reduces the effort required for future updates or changes.
- **Improved Performance:** Efficient code runs faster and is more responsive.

Tools and Practices to Improve Code Quality:

- **Code Reviews:** Peer reviews help identify potential issues and ensure that best practices are followed.
- **Linting Tools:** Tools like ESLint, Pylint, or Checkstyle automatically check for syntax errors, coding standards, and potential bugs.

- **Automated Testing:** Unit tests, integration tests, and regression tests help catch bugs early in the development process.
- **Continuous Integration (CI):** CI pipelines help ensure that every new code change is tested and validated automatically before being integrated into the main project.

Code Metrics

Code metrics are quantitative measures used to assess the quality of a software system. They help in evaluating the maintainability, complexity, efficiency, and reliability of code. By tracking these metrics, development teams can identify areas for improvement and ensure that the code meets performance and quality goals.

Common Code Metrics:

1. **Lines of Code (LOC)**
 - o **Definition:** Measures the number of lines in a program's source code.
 - o **Use:** Often used to gauge the size of a project. More lines can indicate greater complexity, but it's not necessarily a sign of better quality.
 - o **Limitations:** It does not reflect the quality or efficiency of the code. A smaller codebase can be more efficient and maintainable.
2. **Cyclomatic Complexity**
 - o **Definition:** Measures the number of linearly independent paths through a program's source code, often used to assess the complexity of a function or method.
 - o **Use:** Helps in understanding how difficult a piece of code is to test or maintain. Higher values mean more complex, error-prone code.
 - o **Recommended Range:** A cyclomatic complexity of 1-10 is generally considered simple and maintainable. Values over 20 can indicate that the code should be refactored.
3. **Code Coverage**
 - o **Definition:** Measures the percentage of the codebase that is tested by automated tests (unit tests, integration tests, etc.).
 - o **Use:** High code coverage ensures that the critical parts of the code are tested, reducing the likelihood of undetected bugs.
 - o **Ideal Target:** Aim for at least 70-80% coverage, but remember that 100% coverage doesn't guarantee bug-free code.
4. **Coupling**
 - o **Definition:** Measures the degree of dependency between modules or components in a software system.
 - o **Low Coupling:** Indicates that modules are independent, making the system easier to maintain and update.
 - o **High Coupling:** Increases the risk of introducing bugs when making changes, as many components are interdependent.
5. **Cohesion**

- **Definition:** Measures how closely related the responsibilities of a single module are.
 - **High Cohesion:** Means that a module has a single, well-defined purpose, making it easier to maintain and understand.
 - **Low Cohesion:** Suggests that the module does too many unrelated things, making it harder to work with and more prone to bugs.
6. **Maintainability Index**
- **Definition:** A composite metric that measures how easy it is to maintain code. It is calculated based on cyclomatic complexity, lines of code, and code comments.
 - **Use:** Helps developers understand which parts of the codebase are easy to maintain and which need refactoring.
 - **Scale:** Ranges from 0 (hard to maintain) to 100 (easy to maintain). A score below 20 often indicates that the code is difficult to maintain.
7. **Halstead Metrics**
- **Definition:** These metrics focus on the number of operators and operands in the code to measure the complexity, volume, and difficulty of a program.
 - **Halstead Volume:** Measures the total size of the code.
 - **Halstead Difficulty:** Assesses how difficult the code is to write or understand.
 - **Halstead Effort:** Predicts the effort required to implement or maintain the code.
 - **Use:** Helps assess cognitive complexity and the effort needed to maintain the code.
8. **Technical Debt**
- **Definition:** Measures the implied cost of additional rework caused by choosing a quick and easy solution instead of a better, more complex one.
 - **Use:** High technical debt can indicate that the code requires refactoring, and it may slow down future development efforts.
9. **Duplication**
- **Definition:** Measures the amount of repeated code in a project.
 - **Use:** High duplication increases maintenance costs and the likelihood of inconsistencies. Removing duplicates makes the code easier to maintain and update.
10. **Defect Density**
- **Definition:** Measures the number of defects (bugs) in a piece of code relative to its size, usually measured in terms of defects per thousand lines of code (KLOC).
 - **Use:** Indicates the reliability of the code. A lower defect density implies better code quality.

Tools for Measuring Code Metrics:

- **SonarQube:** Provides in-depth analysis of code quality metrics such as code coverage, duplication, and maintainability.
- **Code Climate:** Analyzes complexity, maintainability, and technical debt, and provides a maintainability index.

- **Checkstyle:** A tool for enforcing coding standards and metrics like LOC, cyclomatic complexity, and code style.
- **JDepend:** Measures coupling and cohesion between classes.
- **Coverage.py:** A tool to measure code coverage in Python projects.

Benefits of Code Metrics:

- **Improved Quality:** By regularly tracking code metrics, developers can maintain a high standard of quality.
- **Early Detection of Issues:** Metrics can help spot potential issues like high complexity or poor test coverage early in development.
- **Better Decision Making:** Teams can make informed decisions about when to refactor, optimize, or add tests based on the metrics.
- **Efficient Resource Allocation:** Understanding the complexity and maintainability of code can help in prioritizing work, whether it's bug fixing, refactoring, or feature development.

Code Coverage Analysis

Code coverage analysis measures how much of a software program's source code is executed when a test suite runs. It provides insight into the extent to which the code has been tested, highlighting untested areas. While high coverage doesn't guarantee that the code is bug-free, it increases confidence in the code's robustness and reliability.

Types of Code Coverage:

1. **Function/Method Coverage**
 - **Definition:** Measures whether each function or method in the code has been called by the tests.
 - **Use:** Ensures that all functions or methods are invoked at least once during testing.
2. **Statement Coverage**
 - **Definition:** Measures the percentage of executed statements (or lines of code).
 - **Use:** Helps ensure that each line of code is executed at least once.
 - **Goal:** Typically aim for 70-90%, though 100% is ideal but not always necessary.
3. **Branch Coverage**
 - **Definition:** Measures whether every possible branch (e.g., `if` and `else` conditions) has been tested.
 - **Use:** Ensures that all branches of conditional statements (like `if`, `else`, `switch`, etc.) are tested.
 - **Goal:** Ensuring that both true and false conditions are tested for every decision point.
4. **Condition Coverage**
 - **Definition:** Tests all boolean expressions to ensure that every condition in a branch has been tested.

- **Use:** More fine-grained than branch coverage, as it ensures that each boolean sub-expression in a condition is tested independently.
- 5. **Loop Coverage**
 - **Definition:** Tests whether loops in the code have been executed.
 - **Use:** Ensures that loops run through all possible scenarios (e.g., zero times, once, multiple times).
- 6. **Path Coverage**
 - **Definition:** Measures whether all possible execution paths in the code have been tested.
 - **Use:** Ensures that every potential flow of execution through the code has been tested, but it can be complex for large codebases.

Why Code Coverage is Important:

1. **Identifying Untested Code:** It reveals portions of the code that haven't been exercised by the test suite, allowing developers to add tests for those areas.
2. **Improving Code Quality:** Testing more of the code ensures that it functions correctly under a variety of conditions and reduces the likelihood of bugs.
3. **Assessing Test Suite Effectiveness:** High code coverage shows that the test suite thoroughly examines the code, whereas low coverage may indicate that the test suite is insufficient.
4. **Reducing Technical Debt:** Higher coverage can lead to more maintainable code because it ensures that future changes won't introduce unnoticed bugs in untested areas.

Code Coverage Tools:

- **JUnit + JaCoCo:** For Java projects, JUnit is used for unit tests, and JaCoCo (Java Code Coverage) provides detailed coverage reports.
- **pytest + Coverage.py:** For Python projects, pytest is a popular testing framework, and Coverage.py measures code coverage.
- **Mocha + Istanbul:** For JavaScript projects, Mocha is a testing framework, and Istanbul generates code coverage reports.
- **Cobertura:** A Java-based tool that can generate detailed coverage reports.
- **Visual Studio:** Offers built-in code coverage tools for .NET projects.

Code Coverage Goals:

- **80% Coverage Rule:** While achieving 100% code coverage is ideal, in many cases, aiming for around 80% is considered a good balance between test thoroughness and effort. Going beyond this point can lead to diminishing returns.
- **Focus on Critical Code:** It's important to prioritize coverage for mission-critical code or code that is prone to failure. Not all code needs the same level of coverage (e.g., simple getters/setters or configuration files).
- **Risk-Based Coverage:** Focus coverage efforts on areas of the code that are high-risk or complex.

Limitations of Code Coverage:

1. **False Confidence:** High coverage does not guarantee the absence of bugs. It only ensures that lines or paths of code were executed, not that they are logically correct.
2. **Quality of Tests:** Coverage metrics don't account for the quality of tests. Poorly written tests may still achieve high coverage but fail to catch critical bugs.
3. **Time-Consuming:** Writing tests for every possible path or condition can be time-consuming, especially in large or complex codebases.
4. **Edge Cases:** Code coverage tools might not cover certain edge cases unless explicitly written into the test cases.

Best Practices for Code Coverage Analysis:

1. **Start with Key Areas:** Focus on the most important parts of the code first, especially the core logic and high-risk areas.
2. **Automated Testing:** Integrate code coverage into the Continuous Integration (CI) pipeline to ensure that coverage metrics are continuously monitored.
3. **Improve Incrementally:** Don't aim for 100% coverage from the start. Gradually increase coverage as the test suite evolves.
4. **Test Driven Development (TDD):** Following TDD can naturally lead to higher code coverage, as tests are written before the code itself.
5. **Refactor Untested Code:** If certain parts of the code are difficult to test, consider refactoring them to improve testability.

Code Smells and Anti-Patterns

Code smells and anti-patterns are indicators of poor design or coding practices that can make a system harder to maintain, extend, or understand. While not necessarily bugs or errors, they often signal underlying problems that, if left unaddressed, can lead to more serious issues in the future. Understanding and addressing these issues is crucial for maintaining code quality over time.

Code Smells

A *code smell* is a surface indication that something may be wrong in the code. It doesn't necessarily break the code but points to deeper issues in design or structure that could lead to problems down the line.

Common Code Smells:

1. **Long Method/Function**
 - **Description:** A method that is too long or complex, making it hard to understand and maintain.
 - **Solution:** Break it down into smaller, more manageable methods (e.g., "Extract Method").
2. **Duplicated Code**

- Description: The same or very similar code appears in multiple places.
- Solution: Refactor the code to remove duplication, possibly by using abstraction, inheritance, or utility functions.

3. Large Class

- Description: A class that has too many responsibilities or is bloated with too much code.
- Solution: Break the class into smaller, more cohesive classes (Single Responsibility Principle).

4. Primitive Obsession

- Description: Overuse of primitive data types (like `int`, `string`, `float`) instead of using objects to represent entities or concepts.
- Solution: Use small, specialized classes to encapsulate related functionality (e.g., `Money` class instead of `int` for currency).

5. Feature Envy

- Description: A method in one class is more interested in the details of another class than its own.
- Solution: Move the method to the class that it depends on the most.

6. Data Clumps

- Description: Groups of variables that are often found together in various parts of the code (e.g., several methods that pass around the same three parameters).
- Solution: Encapsulate them into an object, which reduces the number of parameters and provides better encapsulation.

7. Inconsistent Naming

- Description: Variables, functions, or classes have non-descriptive or inconsistent names.
- Solution: Use meaningful, consistent naming conventions to improve code readability.

8. Lazy Class

- Description: A class that does too little or is redundant.
- Solution: Eliminate or merge it with another class to simplify the design.

9. Speculative Generality

- Description: The code contains constructs that are added for future needs that don't exist yet, leading to unnecessary complexity.
- Solution: Simplify the code by removing features that aren't currently required (YAGNI - "You Aren't Gonna Need It").

10. Switch Statements

- Description: Excessive use of switch statements or if-else chains to control flow, often based on type or state.

- Solution: Use polymorphism or the Strategy pattern to eliminate the need for these control structures.
-

Anti-Patterns

An *anti-pattern* is a common solution to a problem that is ineffective and counterproductive over time. Anti-patterns typically arise when a developer tries to apply a solution that seems reasonable but leads to technical debt and long-term issues.

Common Anti-Patterns:

1. God Object / God Class

- Description: A class that knows too much or does too many things, becoming a central hub for multiple responsibilities.
- Solution: Apply the Single Responsibility Principle and break the class into smaller, more focused components.

2. Spaghetti Code

- Description: Code that is poorly structured and tangled, often with excessive interdependencies and unclear flow.
- Solution: Refactor the code into a more modular and clear structure, possibly introducing design patterns or better separation of concerns.

3. Shotgun Surgery

- Description: A small change in one part of the system requires changes in many other parts of the code.
- Solution: Consolidate related behavior into a single location and reduce dependencies by adhering to Separation of Concerns and Encapsulation principles.

4. Golden Hammer

- Description: Over-reliance on a single technology or design pattern to solve all problems, even when it's not the best fit.
- Solution: Be flexible in choosing the right tools and patterns for each problem.

5. Magic Numbers

- Description: Using hard-coded numbers in the code without explanation.
- Solution: Replace magic numbers with named constants to make the code more readable and maintainable.

6. Poltergeist (Ghost) Objects

- Description: Classes that exist only to pass information between other classes and have little to no functionality.
- Solution: Eliminate these classes and pass the data directly or refactor the design.

7. Over-Engineering

- **Description:** Adding more complexity to the code than is necessary, often to prepare for future scenarios that may never happen.
- **Solution:** Follow the KISS principle (Keep It Simple, Stupid) and avoid building unnecessary features or abstraction layers.

8. Copy-Paste Programming

- **Description:** Reusing code by copying and pasting it into different locations rather than creating reusable functions or classes.
- **Solution:** Refactor duplicated code into functions or classes to promote reuse and reduce the risk of bugs.

9. Anemic Domain Model

- **Description:** A domain model where the business logic is placed outside of the entities, leaving them as mere data containers.
- **Solution:** Move the logic into the domain entities themselves to ensure better encapsulation and object-oriented design.

10. Cargo Cult Programming

- **Description:** Including code or structures in a project without fully understanding them, simply because they worked in another context.
- **Solution:** Ensure a full understanding of the code and technologies you use, and avoid blindly copying patterns or frameworks.

Strategies for Addressing Code Smells and Anti-Patterns:

1. **Refactoring:** Regularly refactor code to remove smells and anti-patterns. Small, incremental refactoring can prevent code from becoming unmanageable.
2. **Code Reviews:** Peer code reviews help identify potential code smells and anti-patterns early, allowing teams to address issues before they escalate.
3. **Adopt Design Patterns:** Use established design patterns like Singleton, Factory, or Observer to solve common problems and avoid anti-patterns.
4. **Automated Testing:** Ensure that the system has adequate unit tests and integration tests before refactoring to avoid introducing new bugs.
5. **Follow SOLID Principles:** Adhering to principles like Single Responsibility, Open/Closed, and Dependency Inversion helps create maintainable and scalable code.
6. **Continuous Integration:** Use CI pipelines to detect and fix issues early by incorporating automated linting and code analysis tools (e.g., SonarQube, CodeClimate).

Code Readability and Commenting Best Practices

Code readability is essential for ensuring that software can be easily understood, maintained, and extended by developers. Clean, readable code reduces the likelihood of errors, simplifies

collaboration, and makes debugging more efficient. While comments can provide valuable context, writing readable code should always be the primary focus before relying on comments.

Code Readability Best Practices

1. Meaningful Naming Conventions

- **Use Descriptive Names:** Variables, functions, classes, and other identifiers should have meaningful names that clearly describe their purpose.
 - **Example:** `int totalItemsSold;` is better than `int x;`
- **Avoid Abbreviations:** Unless universally understood (e.g., `URL`, `ID`), avoid abbreviations that may confuse other developers.
- **Consistency in Naming:** Use consistent naming patterns throughout the codebase (e.g., camelCase for variables and PascalCase for class names).

2. Keep Functions/Methods Small

- **Single Responsibility:** Functions should perform a single task. This makes them easier to understand, test, and debug.
- **Limit Function Length:** Aim for functions that fit within a single screen (20-30 lines). Large functions should be refactored into smaller, more modular ones.

3. Use Proper Indentation

- **Consistent Indentation:** Use consistent indentation to clearly define the structure of your code. Typically, 2 or 4 spaces are used for indentation, but the important thing is consistency across the project.
- **Avoid Deep Nesting:** Minimize deeply nested loops or conditionals, as they make the code harder to follow. Consider refactoring using functions to simplify logic.

4. Avoid Magic Numbers and Strings

- **Define Constants:** Replace hardcoded numbers and strings with named constants or enumerations.
 - **Example:** Instead of `if (discount == 10)`, use `if (discount == STANDARD_DISCOUNT)`.
- **Clarify Intent:** Use constants to make the purpose of numbers and values clear to anyone reading the code.

5. Modular Code and DRY (Don't Repeat Yourself)

- **Break Code into Functions/Classes:** Organize code into reusable functions and classes to reduce duplication and enhance maintainability.
- **Reuse Code:** Avoid duplicating logic in different parts of the code. Refactor common functionality into shared components.

6. Use Whitespace Judiciously

- **Separate Logical Blocks:** Use blank lines to separate different sections of code, such as between variable declarations and logic or between functions.
- **Compact Where Necessary:** Don't overuse whitespace in a way that makes the code unnecessarily long or difficult to follow.

7. Readable Conditionals and Loops

- **Descriptive Conditions:** Use descriptive conditions that clearly state what the logic is checking.
 - **Example:** `if (isUserLoggedIn)` is better than `if (loggedIn == true)`.
- **Avoid Complex Conditions:** Break down complex conditional logic into smaller, more understandable parts by using helper functions or variables.

8. Use Proper Error Handling

- **Descriptive Error Messages:** Ensure that error messages clearly describe the issue to help with debugging and future maintenance.
- **Handle Exceptions Gracefully:** Don't leave large sections of code in try-catch blocks without specific exception handling. Handle errors at the appropriate level.

Commenting Best Practices

While writing self-explanatory code is the goal, comments are still necessary to provide additional context, explain complex logic, or document non-obvious decisions.

1. Write Self-Explanatory Code First

- **Minimize the Need for Comments:** Strive to write code that is easy to understand on its own. If the code is clear, the need for comments is reduced.
- **Example:** Use clear variable and function names so that comments are not needed to explain them.

2. Comment Why, Not What

- **Explain Intent or Decisions:** Use comments to explain the reasoning behind a specific implementation or decision, rather than what the code does.
- **Example:** Instead of `// Increment counter`, use `// Increment counter to track user logins within session`.

3. Avoid Obvious Comments

- **Don't State the Obvious:** Comments should add value. Don't comment on things that are self-explanatory from the code.
- **Example:** `// Set x to 10` above `x = 10;` is redundant and unnecessary.

4. Keep Comments Up to Date

- **Update Comments:** Ensure that comments reflect any changes made to the code. Outdated comments are worse than no comments as they can mislead developers.
- **Regular Review:** During code reviews, ensure that comments are aligned with the current implementation.

5. Use Block Comments for Complex Logic

- **Block Comments:** Use block comments (multi-line) to explain complex logic, algorithms, or non-obvious parts of the code.
- **Placement:** Place block comments directly above the code they describe.
- **Example:**

```
java
Copy code
/*
 * This function calculates the factorial of a number.
 * It uses a recursive approach to break the problem
 * into smaller subproblems.
 */
int factorial(int n) {
    if (n == 1) return 1;
    return n * factorial(n - 1);
}
```

6. Use Inline Comments Sparingly

- **Inline Comments:** Use inline comments to clarify a single line or a specific section of code. They should not disrupt the flow of the code.
- **Placement:** Inline comments should be brief and placed on the same line as the code they describe.
- **Example:**

```
java
Copy code
int result = factorial(n); // Recursive call to calculate
factorial
```

7. Comment TODOs and Fixes

- **TODO Comments:** Use `TODO` comments to mark areas of the code that need further work or improvements in the future.
- **FIXME Comments:** Use `FIXME` to indicate code that is broken or needs fixing.
- **Example:**

```
java
Copy code
// TODO: Refactor this method to improve efficiency
// FIXME: This calculation needs validation
```

8. Document Public APIs and Libraries

- **API Documentation:** For public-facing methods, classes, or APIs, use structured documentation comments (e.g., Javadoc, docstrings) to describe usage, parameters, and return values.
- **Example (Javadoc):**

```
java
Copy code
/**
 * Calculates the area of a rectangle.
 *
 * @param width the width of the rectangle
 * @param height the height of the rectangle
 * @return the area of the rectangle
 */
public int calculateArea(int width, int height) {
    return width * height;
}
```

9. Use Version Control History for Context

- **Avoid Over-Documenting:** Some changes, such as tracking when a line of code was added, can be better handled by version control systems (e.g., Git) rather than comments in the code. Don't clutter the codebase with version history comments.

Commenting Tools and Conventions

1. **Use Commenting Conventions:** Adopt a consistent commenting style throughout the project (e.g., Javadoc for Java, docstrings for Python, XML comments for C#).
2. **Leverage IDE Features:** Many IDEs support auto-generating comments for methods, classes, and functions, which can save time and ensure consistency.
3. **Linting Tools:** Tools like ESLint or Pylint can enforce proper code and comment formatting standards to ensure consistency.

Summary of Best Practices:

- Prioritize code readability first, then add comments for additional clarity.
- Write self-explanatory code to reduce the need for comments.
- Use descriptive names for variables and functions to make the code intuitive.
- Comment why something is done, not what the code is doing.
- Keep comments up to date to reflect code changes.
- Avoid unnecessary or obvious comments.
- Use comments to provide context or explain complex logic.

Refactoring Techniques for Code Improvement

Refactoring involves restructuring existing code without changing its external behavior to improve readability, maintainability, and performance. It helps clean up technical debt, enhance code quality, and make future changes easier. Here are common refactoring techniques used to improve code.

1. Extract Method

- **Description:** Breaks down long or complex methods into smaller, more focused methods.
- **Goal:** Improves readability and reusability.
- **Example:**

```
java
Copy code
// Before refactoring
void processOrder() {
    // Code for calculating total price
    // Code for updating inventory
    // Code for sending confirmation email
}

// After refactoring
void processOrder() {
    calculateTotalPrice();
    updateInventory();
    sendConfirmationEmail();
}
```

2. Inline Method

- **Description:** If a method's body is as clear as its name and only called in one place, it can be replaced with the method body directly.
- **Goal:** Simplifies the code when a method is unnecessary.
- **Example:**

```
java
Copy code
// Before refactoring
int getDiscountedPrice(int price) {
    return price * 0.9;
}

int price = getDiscountedPrice(originalPrice);

// After refactoring
int price = originalPrice * 0.9;
```

3. Extract Class

- **Description:** Moves related fields and methods from a large class to a new class to follow the Single Responsibility Principle.
- **Goal:** Reduces class size and improves cohesion.
- **Example:**

```

java
Copy code
// Before refactoring
class Customer {
    String name;
    String address;
    String phone;
    void updateAddress(String newAddress) {
        // logic
    }
}

// After refactoring
class Customer {
    String name;
    ContactInfo contactInfo;
}

class ContactInfo {
    String address;
    String phone;
    void updateAddress(String newAddress) {
        // logic
    }
}

```

4. Inline Temp (Variable)

- **Description:** Replaces temporary variables with direct expressions when the variable doesn't simplify the code.
- **Goal:** Reduces unnecessary variables to improve clarity.
- **Example:**

```

java
Copy code
// Before refactoring
double basePrice = quantity * itemPrice;
if (basePrice > 1000) { /* logic */ }

// After refactoring
if (quantity * itemPrice > 1000) { /* logic */ }

```

5. Replace Temp with Query

- **Description:** If a temporary variable holds the result of an expression, replace it with a method or function call.
- **Goal:** Avoids redundant calculations and makes the code easier to maintain.
- **Example:**

```

java
Copy code
// Before refactoring
double basePrice = quantity * pricePerItem;
if (basePrice > 1000) { /* logic */ }

```

```
// After refactoring
if (calculateBasePrice() > 1000) { /* logic */ }

double calculateBasePrice() {
    return quantity * pricePerItem;
}
```

6. Replace Conditional with Polymorphism

- **Description:** Replaces long `if-else` or `switch` statements that check an object's type or state with polymorphism using inheritance or interfaces.
- **Goal:** Enhances code scalability and readability by removing complex conditional logic.
- **Example:**

```
java
Copy code
// Before refactoring
if (employeeType == "Manager") {
    calculateManagerSalary();
} else if (employeeType == "Engineer") {
    calculateEngineerSalary();
}

// After refactoring
class Employee {
    abstract void calculateSalary();
}

class Manager extends Employee {
    void calculateSalary() { /* logic for manager */ }
}

class Engineer extends Employee {
    void calculateSalary() { /* logic for engineer */ }
}
```

7. Move Method

- **Description:** Moves a method to the class where it logically belongs, typically the class it most interacts with.
- **Goal:** Improves code organization and reduces coupling between classes.
- **Example:**

```
java
Copy code
// Before refactoring
class Customer {
    Order order;
    void calculateOrderTotal() {
        order.calculateTotal();
    }
}

// After refactoring
```



```
class Order {
    void calculateTotal() { /* logic */ }
}
```

8. Rename Method or Variable

- **Description:** Changes a method, variable, or class name to something more meaningful and descriptive.
- **Goal:** Improves readability and understanding of the code.
- **Example:**

```
java
Copy code
// Before refactoring
int a = calculate();

// After refactoring
int totalSales = calculateTotalSales();
```

9. Remove Dead Code

- **Description:** Eliminates unused or unnecessary code, such as methods, variables, or condition branches that are never executed.
- **Goal:** Reduces clutter and prevents confusion.
- **Example:**

```
java
Copy code
// Before refactoring
void calculatePrice() {
    if (isHoliday()) {
        // discount code
    }
    // old discount code (commented or unused)
}

// After refactoring
void calculatePrice() {
    if (isHoliday()) {
        // discount code
    }
}
```

10. Replace Magic Numbers with Constants

- **Description:** Replaces hard-coded values with named constants to give the numbers context.
- **Goal:** Improves readability and maintainability by making the code's intent clear.
- **Example:**

```
java
Copy code
// Before refactoring
double interestRate = balance * 0.05;
```

```
// After refactoring
final double INTEREST_RATE = 0.05;
double interestRate = balance * INTEREST_RATE;
```

11. Introduce Parameter Object

- **Description:** When a method has several parameters that are often passed together, group them into a single object.
- **Goal:** Simplifies method signatures and increases code clarity.
- **Example:**

```
java
Copy code
// Before refactoring
void createOrder(String customerName, String customerAddress, String
customerPhone) { /* logic */ }

// After refactoring
class Customer {
    String name;
    String address;
    String phone;
}
void createOrder(Customer customer) { /* logic */ }
```

12. Decompose Conditional

- **Description:** Breaks down complex conditional expressions into simpler methods.
- **Goal:** Makes conditionals easier to read and maintain.
- **Example:**

```
java
Copy code
// Before refactoring
if (date.before(SUMMER_START) || date.after(SUMMER_END)) {
    charge = winterRate;
}

// After refactoring
if (isWinter(date)) {
    charge = winterRate;
}

boolean isWinter(Date date) {
    return date.before(SUMMER_START) || date.after(SUMMER_END);
}
```

13. Encapsulate Field

- **Description:** Converts public fields into private ones with getter and setter methods.
- **Goal:** Enforces encapsulation, making the class easier to maintain.
- **Example:**

```
java
Copy code
// Before refactoring
public String name;

// After refactoring
private String name;
public String getName() { return name; }
public void setName(String name) { this.name = name; }
```

14. Replace Type Code with Subclasses or Strategy

- **Description:** Replaces type codes (such as enums or constants that indicate different types) with subclasses or the Strategy pattern.
- **Goal:** Improves flexibility and removes rigid conditional logic.
- **Example:**

```
java
Copy code
// Before refactoring
class Employee {
    int type; // 1 = Manager, 2 = Engineer
}

// After refactoring
class Employee {
    abstract void calculateSalary();
}

class Manager extends Employee {
    void calculateSalary() { /* logic */ }
}

class Engineer extends Employee {
    void calculateSalary() { /* logic */ }
}
```

Best Practices for Refactoring

1. **Refactor Incrementally:** Small, consistent refactoring steps help avoid introducing new bugs and make it easier to track changes.
2. **Use Unit Tests:** Ensure the system has adequate unit tests in place before refactoring, so that the behavior remains unchanged. Automated testing helps verify the correctness after refactoring.
3. **Don't Mix Refactoring with New Features:** Refactor in isolation from adding new features. This prevents accidental introduction of bugs and ensures focus on code improvement.
4. **Continuous Refactoring:** Regularly refactor code as part of the development process to prevent the accumulation of technical debt.

5. **Focus on High-Risk Areas:** Refactor areas of the code that are frequently modified or where bugs tend to occur.